AI OPERATIONS (AIOps) · MODULE 4

Serving with Ray Serve

Module 4 · Deck 3 of 4
Deployments · replicas · a canary router · autoscaling · self-healing — CPU only

Third-year Data Science · CPU-only · all code runs on a laptop

Learning outcomes for this deck

Turn a Python class into a scalable Serve deployment with fractional CPUs

Compose a router → model graph with a weighted canary split

Scale replicas manually and automatically, and read the results honestly

Observe self-healing when a replica is killed

Run a promote / rollback drill by redeploying the router

Choose between Ray Serve, MLflow serve and Kubernetes for a given need

Why Ray Serve after MLflow?

What MLflow serve gave us

One process, one model version, restart to change it. Great for dev, tests and containers.

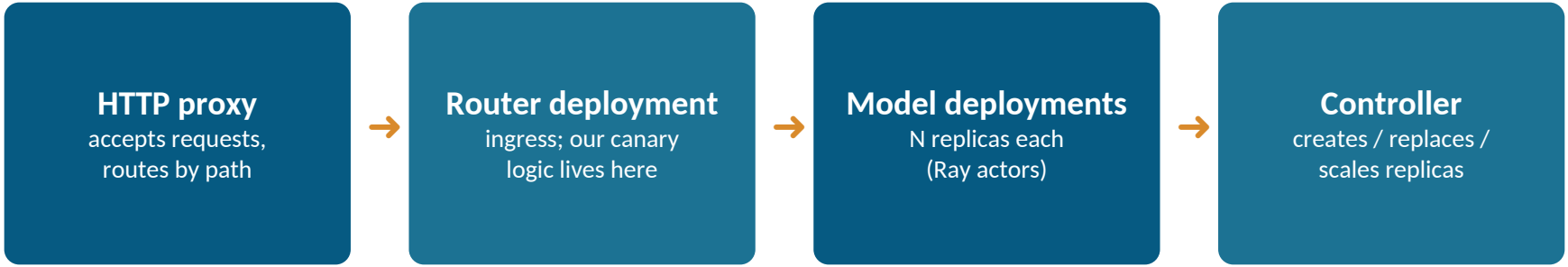
What we still need

Several replicas, traffic split between versions, autoscaling, request routing/batching, multi-model pipelines — in code.

Ray Serve's answer

A serving library on Ray: deployments = replicated actors, composable with Python handles, scaled by a controller. No containers required.

Ray Serve architecture (what runs where)



The controller reconciles desired state (num_replicas, autoscaling_config) with actual actors — the same declarative loop you will meet in Kubernetes (Deck 4). Replicas reserve CPU from the Ray cluster: fractional values such as 0.25 let a laptop host many replicas.

1

Deployments

A decorated class is a scalable service

The ChurnModel deployment

```python
@serve.deployment(
    num_replicas=2, ray_actor_options={"num_cpus": 0.25},
    max_ongoing_requests=16,
    health_check_period_s=5,
    logging_config={"log_level": "WARNING"})
class ChurnModel:
    def __init__(self, path: str, extra_work_ms: float = 0.0):
        self.extra = extra_work_ms / 1000.0
        b = joblib.load(path)
        self.model, self.version = b["model"], b["version"]
        self.replica = serve.get_replica_context().replica_id.unique_id
    async def __call__(self, rows: List[Dict]):
        X = pd.DataFrame(rows)[FEATURES]
        p = self.model.predict_proba(X)[:, 1]
        if self.extra: time.sleep(self.extra)
        return {"predictions": [int(x >= .5) for x in p], "probabilities": [round(float(x), 4) for x in p],
                "model_version": self.version, "served_by": self.replica}
    async def check_health(self):
        if self.model is None: raise RuntimeError("model missing")
```

num_replicas: how many copies · num_cpus: fractional reservation · max_ongoing_requests: back-pressure per replica · check_health: raise → replica is restarted.

The deployment options you will actually use

Option	Meaning	Our value
num_replicas	Fixed number of copies (or `autoscaling_config`)	2 (v1), 2 (v2), 1 (router)
ray_actor_options={'num_cpus':...}	CPU reserved per replica (fractions OK)	0.25 / 0.1
max_ongoing_requests	Queue limit per replica → back-pressure	16
autoscaling_config	min/max replicas, target_ongoing_requests, delays	Section 3
health_check_period_s + check_health()	Liveness: failing replicas are replaced	5 s
logging_config	Replica log level	WARNING

The router: FastAPI ingress with a weighted split

```python
api = FastAPI()

@serve.deployment(num_replicas=1, ray_actor_options={"num_cpus": 0.1}, logging_config={"log_level": "WARNING"})
@serve.ingress(api)
class Router:
    def __init__(self, v1: DeploymentHandle, v2: DeploymentHandle, canary_weight: float):
        self.h = {"v1": v1, "v2": v2}
        self.w = canary_weight
    @api.get("/healthz")
    def healthz(self): return {"status": "ok", "canary_weight": self.w}
    @api.post("/v1/models/churn:predict")
    async def predict(self, req: Req):
        arm = "v2" if random.random() < self.w else "v1"
        try:
            return await self.h[arm].remote([c.model_dump() for c in req.instances])
        except Exception as e:
            raise HTTPException(500, f"{arm} failed: {type(e).__name__}")
```

random() < weight → v2: a canary in two lines. The router holds DeploymentHandles to both models; .remote() sends the call to one of that deployment's replicas.

Compose and run the application

```python
def build_app(canary_weight=0.2, v1_replicas=2, v2_replicas=2, extra_work_ms=0.0):
    v1 = ChurnModel.options(name="ChurnModelV1", num_replicas=v1_replicas).bind("data/model_v1.joblib", extra_work_ms)
    v2 = ChurnModel.options(name="ChurnModelV2", num_replicas=v2_replicas).bind("data/model_v2.joblib", extra_work_ms)
    return Router.bind(v1, v2, canary_weight)       # application graph: Router -> (V1, V2)

# in a notebook / script
import ray; from ray import serve
ray.init(num_cpus=2, include_dashboard=False)       # pretend the laptop has 2 CPUs
serve.start(http_options={"host": "127.0.0.1", "port": 8100})
serve.run(build_app(canary_weight=0.2), route_prefix="/", name="default")

# CPU budget: 2x0.25 (v1) + 2x0.25 (v2) + 0.1 (router) = 1.1 of 2 CPUs
serve.status().applications["default"].deployments    # HEALTHY, replica_states
```

Equivalent CLI for a script: `serve run ray_serve_app:app` (not exercised in the notebook).

2

Canary traffic and replica scaling

Measure the split; scale the bottleneck

Measured canary split — weight 0.2, 300 requests

Version	Requests	Share	Distinct replicas that answered
v1	240	80 %	2
v2	60	20 %	2
errors	0	0 %	—

A second run gave 248 / 52 (83 % / 17 %): that IS the noise. 300 samples ⇒ about ±4 points (binomial). Real canaries run until enough traffic has been seen to compare error rate and latency — not just for a fixed time.

Why replica scaling needs a careful experiment

Tiny model on a 2-core laptop: the HTTP proxy, router and load generator saturate the CPUs before replicas matter — a flat curve, which says nothing about Ray

Fix: make each replica's work heavier on purpose: `extra_work_ms=80` (a blocking sleep ≈ a bigger network; no CPU cost)

Prediction: one replica ≤ 1000/80 ≈ 12 req/s; two ≈ twice that, until another stage saturates

Lesson: scale the bottleneck, not just the model; know what you simulated and say so

Manual scaling experiment

```python
out = []
for n in [1, 2, 4]:
    serve.run(build_app(canary_weight=1.0, v1_replicas=1, v2_replicas=n, extra_work_ms=80),
              route_prefix="/", name="default")
    time.sleep(3); load_test.run(URL, "rest", 4, 2, recs, 1)          # warm-up
    r = load_test.run(URL, "rest", 8, 6, recs, 1); r["v2_replicas"] = n; out.append(r)

# measured (8 clients, v2 only, 80 ms extra work per request)
# v2_replicas   rps   p50_ms   p95_ms   speed-up
#     1        9.3    926     1491      1.00
#     2       16.2    512      707      1.74
#     4       19.8    415      612      2.13    <- bends: router/proxy/CPU now limit
```

Throughput rises while replicas are the bottleneck; latency falls because requests queue less. A second full run gave 10.2 → 16.5 → 20.8 req/s (1.62× / 2.04×): same shape, different numbers. On a real multi-node cluster the same knob adds real cores.

3

Autoscaling and self-healing

The controller reacts to load and to failures

Autoscaling configuration

```python
auto = ChurnModel.options(
    name="ChurnAutoscale", num_replicas=None,
    autoscaling_config=dict(
        min_replicas=1, max_replicas=4,
        target_ongoing_requests=1,      # desired in-flight requests per replica
        upscale_delay_s=2,              # scale UP fast ...
        downscale_delay_s=10,           # ... and DOWN slowly (avoid flapping)
        metrics_interval_s=1, look_back_period_s=3))

serve.run(Router.bind(auto.bind("data/model_v1.joblib", 0.0),
                      small_v2.bind("data/model_v2.joblib", 0.0), 0.0),   # 100 % of traffic to v1
          route_prefix="/", name="default")

# load: 12 concurrent clients for 25 s, then idle for ~18 s
# measured:  idle 1 replica -> peak 4 replicas -> back to 1 after the idle period
```

Same philosophy as the Kubernetes HPA (Deck 4): the controller compares ongoing requests per replica to a target, with delays to damp oscillation.

Transition gotcha: deleting an autoscaled app

- In one of our runs, replacing an autoscaled application directly with another `serve.run` was followed by a stall: a KeyError appeared in the controller log and the next cell hung for 400 s (we did not isolate the root cause)
- Workaround used in the notebook (and tested): `serve.delete('default')`, wait ~3 s, then `serve.run(...)`
- General lesson: new framework versions have rough edges — pin versions (ray[serve]==2.59 here), read the controller log, and keep a clean-teardown step in every runbook

Self-healing: kill a replica under traffic

```python
names = [a for a in ray.util.list_named_actors(all_namespaces=True)
         if "ChurnModelV1" in a["name"] and "REPLICA" in a["name"]]
victim = ray.get_actor(names[0]["name"], namespace=names[0]["namespace"])
ray.kill(victim, no_restart=True)                    # simulate a crash

ok = err = 0; t0 = time.time()
while time.time() - t0 < 12:                         # keep sending traffic
    try:
        r = requests.post(URL, json={"instances": recs[:1]}, timeout=5)
        ok += r.status_code == 200; err += r.status_code != 200
    except requests.RequestException: err += 1
    time.sleep(0.1)

# measured: 87 OK, 0 errors; final replica_states = {'RUNNING': 2}   (replacement started)
```

The controller notices the dead actor and creates a replacement; the router avoids unavailable replicas. Same promise as a Kubernetes ReplicaSet — one layer lower.

4

Release engineering with Ray Serve

Promote, roll back, compose

Promote / rollback drill — redeploy the router only (measured)

Stage	Weight to v2	v1 share	v2 share	Errors
canary 20 %	0.2	79 %	21 %	0
canary 50 %	0.5	47 %	53 %	0
PROMOTE 100 %	1.0	0 %	100 %	0
ROLLBACK 0 %	0.0	100 %	0 %	0

Changing the router's argument redeploys ONLY the router: model replicas keep their loaded weights (no cold start), so rollback is as quick as promotion.

Composition: an ensemble deployment (Task A)

```python
@serve.deployment(num_replicas=1, ray_actor_options={"num_cpus": 0.1})
@serve.ingress(api2)
class Ensemble:
    def __init__(self, h1, h2): self.h1, self.h2 = h1, h2
    @api2.post("/v1/models/churn:predict")
    async def predict(self, req: Req):
        rows = [c.model_dump() for c in req.instances]
        a, b = await asyncio.gather(self.h1.remote(rows), self.h2.remote(rows))  # PARALLEL fan-out
        p = [round((x + y) / 2, 4) for x, y in zip(a["probabilities"], b["probabilities"])]
        return {"probabilities": p, "predictions": [int(x >= .5) for x in p],
                "members": [a["model_version"], b["model_version"]]}

serve.run(Ensemble.bind(v1_deployment, v2_deployment), route_prefix="/", name="default")
# measured: ensemble 0.1029  — between v1 (0.0579) and v2 (0.148)
```

Handles make model pipelines ordinary Python: pre-processing → model → post-processing, each independently scaled.

Task B — which arm meets the SLO? (measured, 4 clients)

Arm	Throughput (req/s)	p95 (ms)	Meets p95 ≤ 1.5 × v1?
v1-only (logistic regression)	55.0	98.0	reference
v2-only (random forest)	39.5	152.6	No (1.56×)

Higher accuracy is not free. Options: add replicas (cost), shrink the forest (accuracy), send v2 a smaller canary share, or accept a looser SLO. A second run gave 48.0 vs 33.8 req/s and p95 112.8 vs 171.0 ms (1.52×, still failing). Compare ratios, not absolutes.

Which serving layer when?

Need	MLflow serve	Ray Serve	Kubernetes
Quick local endpoint from the registry	★★★	★★	★
Weighted canary / pipelines in code	—	★★★	★ (replica ratio) / mesh
Autoscaling on request load	—	★★★ (within Ray)	★★★ (HPA)
Process isolation, resource limits, node scheduling	—	★	★★★
Rolling updates, self-healing across machines	—	★★ (Ray cluster)	★★★
No container tooling required	★★★	★★★	—

They compose: Ray Serve and MLflow servers both run happily INSIDE Kubernetes pods.

HANDS-ON

Notebook 4c — Ray Serve (core ≈ 20 min)

1. §1: ray.init(num_cpus=2), deploy the app, inspect replicas with serve.status()
2. §2: send 300 requests; confirm ≈ 20 % reach v2 and that each version is served by 2 replicas
3. §3: scale v2 to 1, 2, 4 replicas with extra_work_ms=80 — where does the curve bend, and why?
4. §4: autoscaling: plot replicas over time during 25 s of load and 18 s idle
5. §5–6: kill a replica under traffic; run the promote/rollback drill
6. Take-home: YOUR TURN A (ensemble deployment) and B (SLO decision)

Notebook: Lecture4c_Ray_Serve_Exercise.ipynb (≈ 6 min to execute end-to-end; use fewer cells in class)

Common errors and fixes

Symptom	Likely cause	Fix
Deployment stuck in DEPLOYING / pending	Requested CPUs exceed ray.init(num_cpus)	Lower num_cpus per replica or replica count
Hangs after replacing an autoscaled app	Seen once; likely controller state for the removed deployment	serve.delete(app) first, wait, then serve.run
Machine slow, odd behaviour on rerun	Zombie Ray processes from an earlier run	ray stop --force; pkill -9 -f raylet; rm -rf /tmp/ray/session_*
No scaling gain with more replicas	Another stage (proxy/router/CPU) is the bottleneck	Profile each stage; scale the bottleneck
Relative path 'data/model_v1.joblib' not found	Replica working dir differs	Start Ray from the notebook folder; or use absolute paths
ModuleNotFoundError in replica	Module not importable in workers	Run from the same folder; or runtime_env={'working_dir': '.'}

Recap — Ray Serve

- @serve.deployment = replicas + resources + back-pressure; fractional CPUs make laptops viable
- A router + handles give weighted canary and pipelines in a few lines of Python
- Replica scaling is bounded by the real bottleneck; autoscaling follows ongoing requests (up fast, down slowly)
- Replicas are supervised: kill one and the controller replaces it
- Promote / rollback = redeploy the router with a new weight

NEXT → Deck 4 — the same ideas (replicas, probes, rollout, autoscaling) as Kubernetes objects on a 3-node minikube